OmniSight: A Multimodal Self-Healing Agent for Resilient UI Automation

Juhi Thacker · Oct 27, 2026

Abstract

OmniSight is a robotic process automation (RPA) agent that keeps completing tasks after the user interface it automates has changed. Conventional bots locate elements through fixed selectors (IDs, CSS classes, XPaths), so a renamed button, a moved element or a pop-up can stop a whole workflow and force manual repair. OmniSight instead combines simplified DOM analysis with a multimodal vision model: it finds the intended element by what it is and how it looks, verifies the screen state after every step, and repairs failed steps through a six-rung healing ladder ordered from the cheapest strategy (retry with a wait) to the most expensive (vision-based re-location and human escalation). Healing is triggered only on failure, successful fixes are stored in a locator memory so the same failure is cheap next time, and every repair is logged with its strategy and a confidence score. The system also accepts a plain-English goal and converts it into an executable plan, and exposes runs, screenshots and healing reports through a dashboard. We describe the design and a controlled evaluation in which ten to fifteen UI mutations are applied to a test web application and three setups (plain bot, DOM-only healing, full OmniSight) are compared on success rate, time per run and cost per run. This document presents the architecture and experimental protocol; results will be added once the benchmark has been run.

Keywords: robotic process automation, self-healing test automation, multimodal models, UI automation, locator memory, web agents

1. Introduction

UI automation is brittle because it depends on implementation details of the interface rather than on what a person sees. A script that clicks `#btn-login` works until a developer renames the ID, restyles the button or places a cookie banner over it. Each such change turns into manual maintenance, which is costly and repetitive for the teams that own the automation.

OmniSight addresses this by treating a UI element as something to be recognised, not just addressed. The agent reads a simplified view of the page (visible interactive elements with text, role and position) and, only when that is not enough, shows a screenshot to a multimodal model and asks where the intended element is. Because calling a vision model costs time and money, it is used strictly as a fallback.

The contributions of this work are:

1. A JSON workflow format and step runner in which elements can be requested by natural-language description rather than by selector.
2. A cost-ordered self-healing ladder that escalates from DOM-level strategies to vision and finally to a human.
3. A locator memory and confidence scoring scheme so that successful repairs are reused and uncertain ones are reviewed.
4. A natural-language planner, safety guardrails and a dashboard that make each run observable and auditable.
5. A benchmark protocol built on a fully controllable test application with a catalogue of ten UI mutations.

2. Problem Statement and Motivation

Selector-based automation fails whenever a website or application update changes an ID, class or XPath. The failure is usually silent until a run breaks, after which someone must find the new selector and edit the script by hand.

The goal of OmniSight is an automation agent that detects a failure, understands the screen the way a person would, finds the intended element again, completes the task, and remembers the fix for next time. Three properties define success:

- **Resilience:** the workflow still completes after cosmetic and structural UI changes.
- **Efficiency:** the expensive vision model runs only when cheaper strategies have failed.
- **Accountability:** every repair is recorded with its strategy, old and new locator, and confidence, so that a human can audit it.

3. Objectives and Research Questions

The system is built to meet eight objectives:

1. Run multi-step workflows defined in a simple JSON format.

2. Locate UI elements by natural-language description, with no hard-coded selector required.
3. Verify after every step that the expected state was reached.
4. Heal broken steps using a ladder of strategies, from cheap DOM methods to vision.
5. Remember successful fixes (locator memory) so the same failure is cheap the next time.
6. Accept a plain-English goal and turn it into an executable plan.
7. Provide a dashboard with run history, screenshot timeline and a healing report.
8. Measure the benefit with a benchmark: plain bot vs DOM healing vs full OmniSight.

The benchmark is designed to answer three research questions:

- **RQ1:** How much more often does a workflow complete under UI mutation when DOM healing is added to a plain bot?
- **RQ2:** What additional mutations does vision-based healing recover that DOM-only healing cannot?
- **RQ3:** What does that extra resilience cost in time and money per run, and how much does locator memory reduce it on repeat runs?

4. Background and Related Work

OmniSight sits between three lines of existing work. This section positions it by category; formal citations for each category still need to be added from the literature before submission.

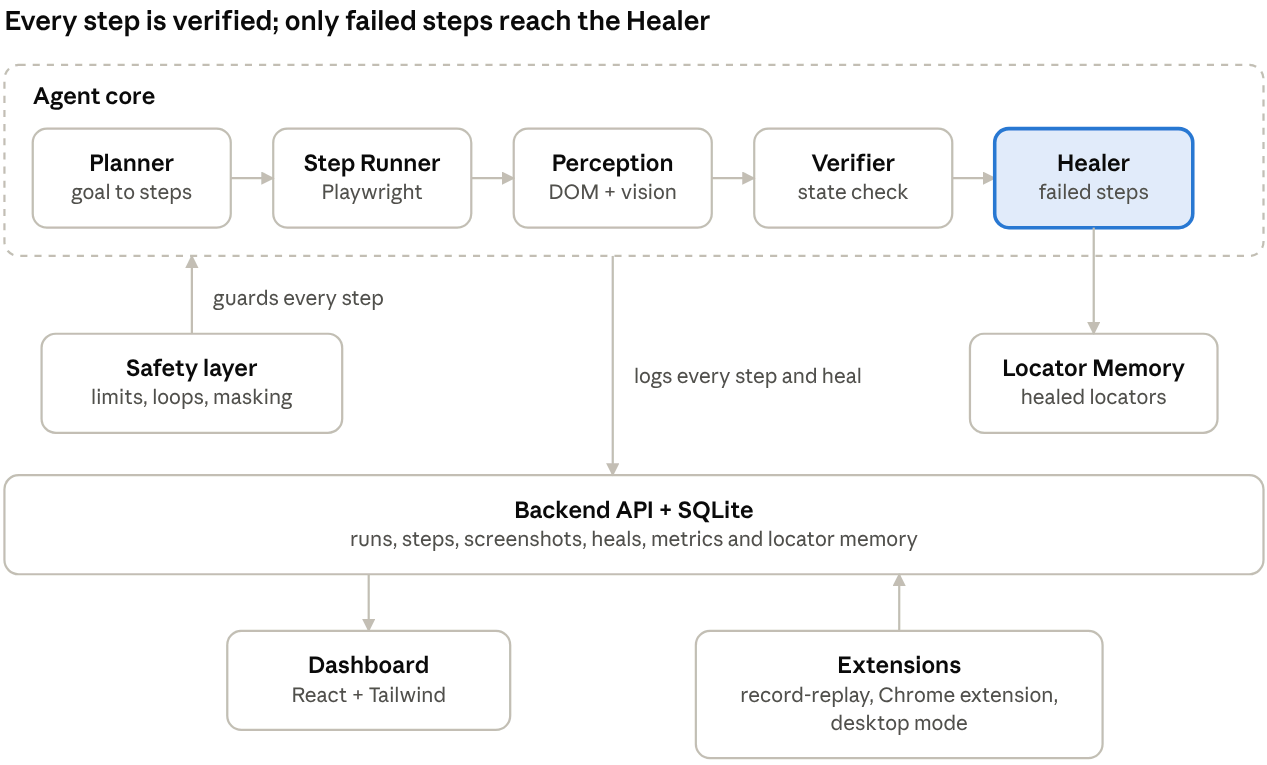
Selector-based browser automation. Frameworks such as Playwright and Selenium drive a browser through locators. Playwright's built-in locators (by role, text and label) and its codegen tool make scripts more robust than raw XPaths, and OmniSight builds on them rather than replacing them. They remain static, though: once a locator stops matching, the script fails.

Self-healing test automation. Self-healing approaches keep alternative attributes for an element and fall back to them, or fuzzy-match against the current DOM, when the primary locator fails. These methods work well for attribute changes but cannot recover when the page structure or appearance changes more deeply, or when a blocker such as a modal hides the target.

Vision and multimodal agents. Multimodal models can read a screenshot and answer where an element is. Used on every step this is slow and expensive. OmniSight's distinguishing choice is to treat vision as one rung on a cost-ordered ladder, invoked only after cheaper strategies fail, and to pair it with step verification, locator memory and human escalation.

5. System Architecture

OmniSight is a pipeline of ten cooperating modules, shown below. The Planner turns a natural-language goal into steps; the Step Runner executes each one with Playwright; Perception builds a simplified DOM (visible interactive elements with text, role and position) and sends a screenshot to the vision model only when needed; the Verifier checks whether the expected state was reached.



OmniSight architecture · 10 modules

When a step fails, the Healer runs its strategy ladder (Section 6) and writes successful fixes to Locator Memory. The Safety layer constrains every step, the backend and SQLite store runs, steps, screenshots, heals and metrics, and the Dashboard shows them. Extensions (record-and-replay, the Chrome extension and desktop mode) feed workflows and runs into the same engine and backend.

6. Self-Healing Methodology

When a step fails verification, the Healer tries strategies in order, cheapest first, and stops at the first one that succeeds:

1. **Retry with a wait:** the element may simply not have rendered yet.
2. **Fallback locators:** try the element's text, aria-label and role.
3. **Fuzzy DOM match:** compare the simplified DOM against the description and pick the closest element.
4. **Vision-based re-location:** send a screenshot and element descriptions to the multimodal model, which returns coordinates or the matching element.
5. **Blocker handling:** detect and dismiss pop-ups, modals, cookie banners and overlays, then retry.
6. **Human escalation:** if everything fails, stop and send a screenshot to a person.

Locator memory. After a successful heal, the old and new locators are stored. On the next run the healed locator is tried first, so a repeated failure costs almost nothing and does not call the vision model again.

Confidence scoring and review. Every heal records the strategy used, the old and new locator, and a confidence score. Low-confidence heals are placed in a human review queue instead of being trusted silently. This keeps the benefit of automatic repair while limiting the risk of a wrong but plausible match.

Heal only on failure. The Verifier runs after each step, but the Healer and the vision model are invoked only when a step fails. This is the central cost-control principle of the design.

7. Safety and Reliability Design

An agent that can repair its own steps can also repair them wrongly, so the safety layer limits what it is allowed to do. It enforces four rules:

Control	Purpose
Maximum step limit per run	Stops a run that keeps acting without finishing
Loop detection	Catches repeated identical states or actions
Confirmation before destructive actions (delete, pay, submit)	Prevents an irreversible wrong click

Control	Purpose
Masking of passwords and sensitive fields in screenshots and logs	Keeps secrets out of stored images and the vision model's input

Together with confidence scoring and the human review queue, these controls keep automatic healing auditable and reversible where possible.

8. Implementation

Technology choices were fixed at the start to avoid rework, and favour installing existing tools over building new ones.

Layer	Choice
Browser automation	Playwright (Node.js, TypeScript planned)
Desktop automation	robotjs (Node) or PyAutoGUI (Python), using the same screenshot-and-vision loop
Vision and planning	Hosted multimodal model API, wrapped in one function so the provider can be swapped
Backend	Express (or FastAPI)
Storage	SQLite for logs, runs and locator memory
Dashboard	React and Tailwind with a component library
Chrome extension	Manifest V3, reusing the backend
Test application	Own app (Shoes Mart planned), so UI mutations are fully controllable

Hardware constraint. Development runs on a laptop with 8 GB of RAM, so no local vision model is used. All vision and planning calls go to a hosted API, which makes cost and latency measurable benchmark outputs.

The repository is organised by module:

```
omnisight/
  engine/       # step runner, workflow JSON
  perception/   # DOM simplifier, vision calls
  healing/      # ladder, locator memory
  planner/
  server/       # Express API + SQLite
  dashboard/    # React
  test-app/     # Shoes Mart or link to it
  docs/         # problem statement, devlog
```

A short daily entry in `docs/devlog.md` records what broke and how it was fixed; it becomes the raw material for the project report.

9. Experimental Design

The evaluation measures how well each setup survives controlled UI changes. Results are not yet available; this section fixes the protocol so that the benchmark can be run and reported without changing the rules afterwards.

Test application. A web app that the author fully controls is essential, because mutations must be applied on demand. Shoes Mart is planned. The demo workflow is: open the site, log in, search for a product and add it to the cart.

Mutations. Each mutation is applied to the test app and is also one benchmark case. They fall into three groups:

#	Mutation	Group
1	Renamed element ID	Attribute change
2	Changed CSS class	Attribute change
3	Changed button text	Attribute change
4	Moved button to a different position	Layout change
5	Changed element order in the layout	Layout change
6	Changed colours and styling	Layout change
7	Wrapped element in a new container	Layout change
8	Pop-up or modal appearing on load	Blocker

#	Mutation	Group
9	Cookie banner covering the page	Blocker
10	Overlay blocking a button	Blocker

The final benchmark will use 10 to 15 mutations.

Compared setups.

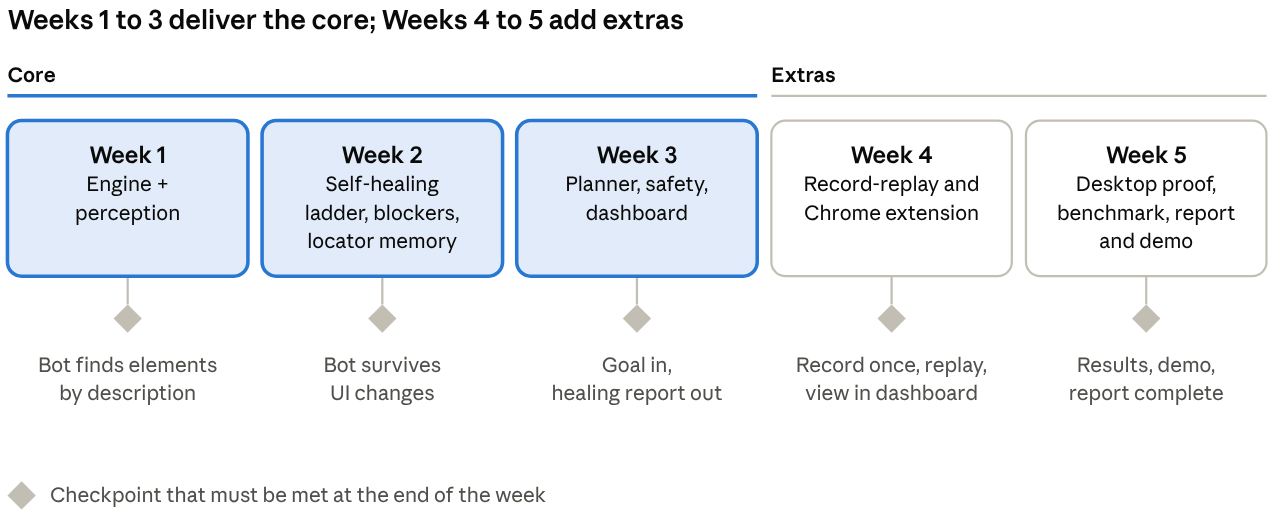
Setup	Description
Plain bot	Fixed selectors only, no healing
DOM healing only	Retry, fallback locators and fuzzy DOM match
Full OmniSight	DOM healing plus vision, blocker handling and locator memory

Metrics. For every setup and mutation: success rate, time per run and cost per run. For full OmniSight the dashboard additionally reports heal rate, average heal time and the share of heals sent to human review.

Procedure. Run the unmodified workflow first to confirm it passes, apply one mutation at a time, run each setup, and record the outcome. Mutations are also introduced from Week 1 of development, not only during the final benchmark, so that healing logic is tested as it is written.

10. Development Methodology and Timeline

The build is phased so that the core system exists by the end of Week 3 and the extras are added afterwards. Each week ends with a checkpoint that must be met before moving on.



five-week plan · 5 checkpoints

The weekly scope is as follows.

- **Week 1, engine and perception:** repository and structure, JSON workflow format, step runner with a screenshot after every step and SQLite logging, then the demo workflow run end to end and broken with the first mutations, followed by the DOM simplifier, vision element finder and verifier.
- **Week 2, self-healing:** DOM-based ladder (retry, fallback locators, fuzzy match), vision recovery, blocker handling, locator memory with confidence scores, the human-escalation path, and a stress test against 10 or more mutations.
- **Week 3, planner, safety and dashboard:** natural-language planner, step limit, loop detection, destructive-action confirmation and masking, the backend API, and the React dashboard with run history, screenshot timeline, healing report, metrics and review queue.
- **Week 4, record-replay and Chrome extension:** record browser actions into JSON workflows with several locators per element (ID, text, role, screenshot crop), then a Manifest V3 extension that records and runs workflows, sends runs to the backend and shows live step status.
- **Week 5, desktop, benchmark and submission:** a thin desktop proof of concept (for example Notepad), the benchmark across 10 to 15 mutations and three setups, then the README with an architecture diagram, a 3 to 5 minute demo video, the project report and a rehearsal.

Seven working rules govern the schedule:

1. Core first, extras last: Weeks 1 to 3 are the project.

2. Heal only on failure.
3. Do not build what can be installed; use Playwright's built-in locators and codegen and a component library for the dashboard.
4. Time-box everything: if a task takes double its slot, move on and return during a buffer.
5. Commit daily and keep the devlog.
6. Test with mutations from Week 1, not only in Week 5.
7. Use an AI coding assistant for boilerplate (UI, CRUD, tests) so personal time goes to the healing logic.

11. Limitations and Threats to Validity

The plan is ambitious and its limits should be stated alongside its claims.

- **Schedule risk.** The scope equals about 30 working days of effort compressed into 35 calendar days, at roughly 5 to 6 focused hours a day including weekends. Skipped days would break the plan.
- **Components likely to overrun.** Desktop support and the Chrome extension carry the most risk. If development is behind at the start of Week 4, simplified versions of both will be built so the full scope still appears in the report.
- **Proof-of-concept status.** Desktop support is deliberately a thin demonstration on one workflow (for example Notepad), and the report must state clearly which parts are proofs of concept.
- **Cost and latency.** Vision calls add money and delay, which is why healing runs only on failure; the benchmark reports this cost rather than hiding it.
- **External validity.** Mutations are applied to one self-built application, so results may not transfer to third-party sites with different structures, dynamic content or anti-automation measures.
- **Internal validity.** Mutations are introduced by the author, who also designs the healer, which can bias the catalogue toward changes the system handles. Reporting every mutation, including failures, reduces this risk.
- **Model dependence.** Vision results depend on the hosted model; wrapping the call in a single function lets the provider be swapped and re-tested.

12. Conclusion and Future Work

OmniSight proposes that UI automation should recover from interface change the way a person would: look at the screen, recognise the intended element, finish the task and remember the fix. Its main design choices are a cost-ordered healing ladder, vision used only on failure, persistent locator memory, and confidence-scored, human-reviewable repairs.

The next steps are to build the core in Weeks 1 to 3, run the mutation benchmark, and report the measured success rate, time and cost per run for each setup. Record-and-replay with multi-locator capture, the Chrome extension and desktop support are planned extensions, and the final report will state which of them are complete and which are proofs of concept.

References and Tools

No literature has been cited yet. Before submission, add references for each category in Section 4 (selector-based automation, self-healing test automation, multimodal and vision-based UI agents).

Tools used in this project: Playwright, Node.js, TypeScript, Express, SQLite, React, Tailwind CSS, robotjs or PyAutoGUI, and a hosted multimodal model API.